



Engenharia de **Custos de LLM** (custo por tarefa, cache, roteamento)

Autor: PDI Marcos Luciano - marcosluciano.rodrigues@v4company.com

Unidade: FV Marketing / V4 Company : Automação & Infraestrutura

Módulo: 04 · Engenharia de IA, RAG e Vetores

Data: Agosto 2026

Status: **NÃO PUBLICADO** · Desenvolvido / Homologação pendente

Versão: 1.0

token / cache / finanças

1. Contexto

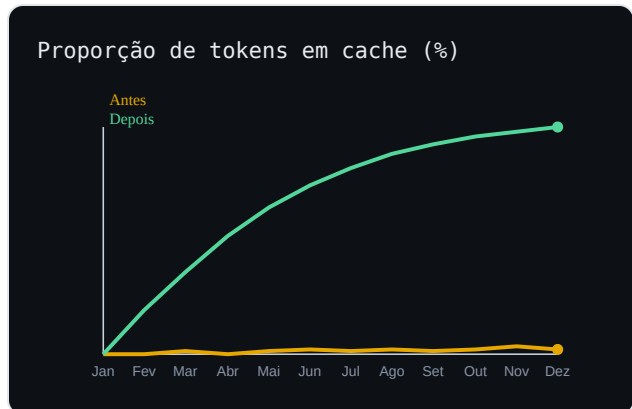
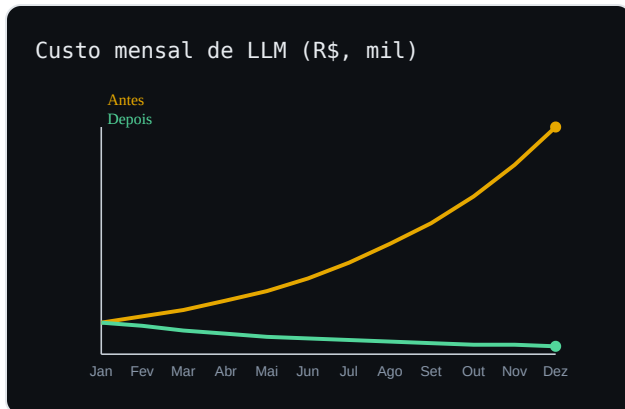
LLM cobra por **token** (entrada + saída). Em produção, o custo explode silenciosamente: prompts repetidos, chamadas redundantes e modelo caro onde um barato bastaria. Reduzir custo não é "usar menos IA" : é **usar a infraestrutura certa**: cache, batch e *model routing*.

ANALOGIA DO BUFFET

Token é a comida no prato : você paga por grama. **Cache semântico** é guardar a sobra na geladeira: se outro pede a mesma coisa, você serve do pote, não cozinha de novo. **Batch** é fazer 100 pratos de uma vez no forno, mais barato que um a um. **Model routing** é mandar o prato simples pro cozinheiro júnior e o complexo pro chef : paga o chef só quando precisa.

2. Diagnóstico

Toda pergunta ia direto pro modelo flagship, sem reaproveitar respostas idênticas e sem agrupar chamadas. A fatura crescia 20% ao mês e ninguém sabia qual fluxo consumia mais. O custo era **opaco e não-atribuído**.



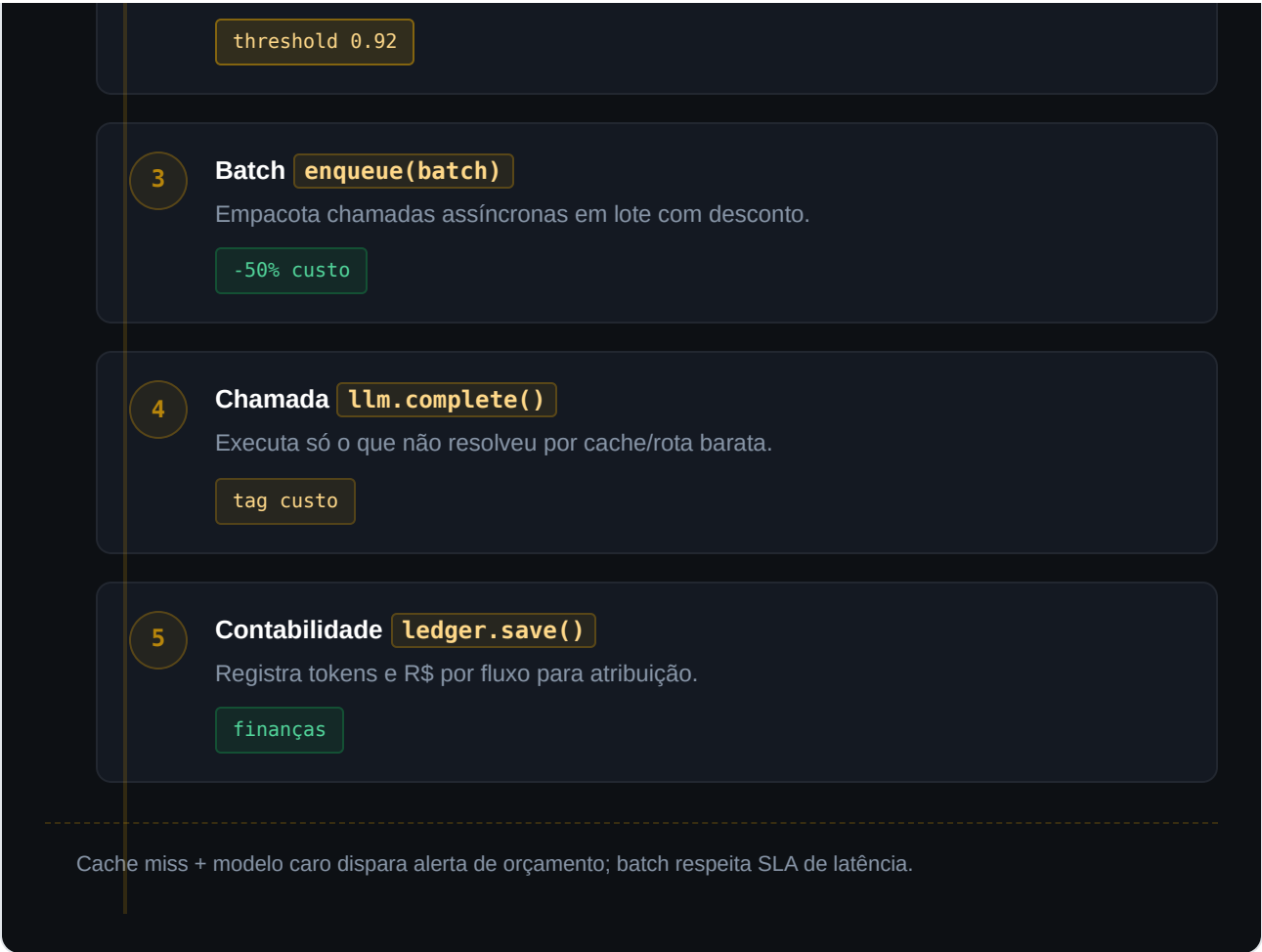
Raiz do problema: 100% das chamadas no modelo caro, sem cache de respostas repetidas e sem roteamento por complexidade. Custo é arquitetura, não acidente.

3. Solução

Trinca de eficiência: (1) **cache semântico** : responde do cache quando a pergunta é similar a uma já respondida; (2) **batch** : empacota chamadas não-urgentes em lotes com desconto; (3) **model routing** : classifica a pergunta e manda pro modelo barato (ex.: mini) ou caro (frontier) conforme a complexidade.

4. Como funciona (pipeline)





5. Antes vs Depois

CENÁRIO	ANTES (FLAGSHIP PURO)	DEPOIS (ROUTING+CACHE+BATCH)
Resposta repetida	Recalculada	Servida do cache
Chamada simples	Modelo caro	Modelo barato
Lote	1 a 1	Batch com desconto
Visibilidade	Nenhuma	Custo por fluxo

6. Entregas

- **Módulo** `model_router.py`: classificação de complexidade e roteamento.
- **Cache** `semantic_cache.py`: cache de respostas por similaridade.
- **Script** `batch_runner.py`: empacotamento de chamadas assíncronas.

- **Dashboard** custo.md: led de tokens e R\$ por fluxo.

7. Métricas



Meta: reduzir o custo mensal em $\geq 85\%$ mantendo qualidade (score ≥ 0.90) via cache + routing + batch.

9. Decisões e tradeoffs

- Roteamento por complexidade em vez de modelo maxi para tudo: troquei simplicidade por governança, porque o maxi custa 10x e a regra trivial vai para leve e complexo vai para forte.
- Cache semântico com meta de hit maior ou igual a 30 por cento e regra de nunca cachear PII: aceitei gestão de invalidação para não pagar 2x a mesma pergunta, sem expor dado sensível.
- Custo por tarefa com meta menor ou igual a baseline vezes 0.4 e ledger por fluxo: escolhi contabilidade visível para permitir precificar ao cliente.
- Budget por cliente com alerta em 80 por cento: preferi travar crescimento de gasto cedo a descobrir estouro na fatura.
- Batch assíncrono respeitando SLA: empacotei chamadas para buscar desconto preservando score maior ou igual a 0.90 e redução maior ou igual a 85 por cento.

10. Impacto no negócio

O framework com custo por tarefa menor ou igual a baseline vezes 0.4, hit de cache maior ou igual a 30 por cento e alerta em 80 por cento do budget torna o agente precificável e reduz o custo mensal em meta maior ou igual a 85 por cento, o que destrava margem e evita subsídio invisível de inferência.

11. Referências de estudo

Trilha de estudo: comece pelo curso, depois o vídeo, e use os docs como consulta de implementação.

- Curso: FinOps for AI and LLM Cost Optimization, plataforma Udemy.
- Vídeo: Redução de custo de LLM com cache e roteamento, plataforma YouTube, canal Y Combinator.
- Doc oficial: Guia de preços e tokens da API, documentação oficial OpenAI.
- Doc oficial: Guia de prompt caching, documentação oficial Anthropic.

8. Status final

NÃO PUBLICADO : Desenvolvido e em homologação. Aguarda revisão antes de produção.